
Claude Certified Architect Foundations (CCAR-F)

A practical study guide based on the official exam blueprint.
Concepts explained in plain English, with architecture examples
and scenario-based practice.

Rupesh Panda

Updated August 2026

START HERE

How to use this guide

The official blueprint is the backbone. The explanations, examples, and practice questions are here to make that blueprint easier to understand and apply.

This guide follows the public Claude Certified Architect - Foundations exam blueprint, version 1.0 effective July 2026. I have kept the official domain structure and objective names, then expanded each objective in plain language. The goal is not to memorize product terms. The goal is to understand what each control does, when to use it, and why one architecture is better than another in a given scenario.

HOW I WOULD STUDY IT

Read one domain. Then close the guide and explain each objective in your own words. After that, answer the practice questions without looking back. For every miss, write down the objective and the exact condition in the question that should have changed your choice.

Contents

1. Exam at a glance and the architectural mindset
2. Domain 1: Agentic Architecture & Orchestration (27%)
3. Domain 2: Tool Design & MCP Integration (18%)
4. Domain 3: Claude Code Configuration & Workflows (20%)
5. Domain 4: Prompt Engineering & Structured Output (20%)
6. Domain 5: Context Management & Reliability (15%)
7. Seven patterns that resolve many difficult questions
8. 25 original scenario-based practice questions
9. Answer key with rationales
10. Exam-day quick scan and 30-objective checklist

Source note: Exam facts and blueprint are based on Anthropic's public CCAR-F materials. Technical behavior was cross-checked against current Claude documentation in August 2026. Product details can evolve, so use the official exam guide for exam scope and current docs for implementation details. All practice questions here are original and are not live exam content.

SECTION 1

Exam at a glance

Before learning individual features, understand what the exam is measuring: architectural judgment under constraints.

ITEM	CURRENT DETAIL
Exam	Claude Certified Architect - Foundations (CCAR-F)
Questions	60
Time	120 minutes, about 135 minutes total seat time
Structure	4 scenarios selected from a published bank of 6; about 15 questions per scenario
Question types	Multiple choice and multiple response
Passing score	720 on a scaled 100-1,000 range
Delivery	Online proctored or Pearson VUE test center
List price	\$125 USD
Credential validity	12 months

The five domains

DOMAIN	WEIGHT	WHAT IT REALLY TESTS
1. Agentic Architecture & Orchestration	27%	Agent loops, subagents, enforcement, decomposition, hooks, sessions
2. Tool Design & MCP Integration	18%	Tool interfaces, errors, tool choice, MCP, built-in tools
3. Claude Code Configuration & Workflows	20%	CLAUDE.md, skills, rules, plan mode, refinement, CI/CD
4. Prompt Engineering & Structured Output	20%	Criteria, examples, schemas, validation, batches, review
5. Context Management & Reliability	15%	Context degradation, escalation, errors, human review, provenance

The six official scenario families

Customer Support Resolution Agent; Code Generation with Claude Code; Multi-Agent Research System; Developer Productivity with Claude; Claude Code for CI/CD; Structured Data Extraction. A live exam form uses four of these six

scenarios, with roughly 15 questions attached to each selected scenario. Prepare all six because the same core mechanisms reappear in different production contexts.

THE EXAM MINDSET

Do not ask, "Which answer sounds the most sophisticated?" Ask, "What is the actual failure mode or requirement, and which option directly addresses it with the least unnecessary complexity?"

EXAM GUIDE VS CURRENT CLAUDE

The certification guide is versioned while Claude Code and the Claude APIs continue to evolve. For exam questions, treat the published CCAR-F guide as the authority on scope and terminology. Use current documentation to understand implementation details. This guide calls out important differences when they could affect an answer.

Mechanics worth memorizing

- ``stop_reason``, ``tool_use``, ``tool_result``, and matching ``tool_use_id`` in the agent loop.
- The exam-guide subagent spawning pattern: ``Task`` must be available to the coordinator, and context must be passed explicitly.
- ``tool_choice``: ``auto`` may answer or call a tool; ``any`` requires some tool call; a specific tool forces that tool.
- PreToolUse vs PostToolUse: prevent before execution; validate or normalize after execution.
- Grep vs Glob vs Bash: file contents, file paths, shell commands.
- CLAUDE.md scopes, plus ``/memory`` for memory/instruction inspection and ``/context`` for broader current context diagnostics.
- Claude Code CI: ``-p`` / ``--print`` for non-interactive execution and structured JSON for machine parsing.
- Message Batches: 50% pricing, up to 24 hours, out-of-order results, and ``custom_id`` correlation.

Five reading habits that save marks

- When the stem says must, never, always, block, or require, look for an enforcement mechanism, not only a prompt reminder.
- When a failure appears after a change, ask what actually changed before redesigning the entire system.
- Distinguish routing from enforcement. A model can choose the right tool and still violate workflow ordering if the workflow is not gated.
- Distinguish syntax from truth. A valid JSON object can still contain the wrong business answer.
- On multiple-response questions, select exactly the number of answers requested. Treat the answer count as part of the question.

A reliable way to read a scenario



Official references checked: [Official certification page](#) | [Claude documentation](#)

Agentic Architecture & Orchestration

WHAT THIS DOMAIN IS REALLY ASKING

Can the system take a goal, decide what happens next, use tools, delegate work when useful, and still remain controlled and observable?

1.1 Agentic Loops

Design and implement agentic loops for autonomous task execution.

What it means: An agent is not a single model call. It is a controlled loop in which Claude can reason, request a tool, receive the result, reason again, and continue until it reaches a stopping condition.

How it works: Your application sends the conversation to Claude and inspects the structured response. When ``stop_reason`` is ``tool_use``, execute the requested tool or tools, append the assistant ``tool_use`` block to history, then append the corresponding ``tool_result`` block in a user message with the matching ``tool_use_id``. Send the updated history back to Claude. ``end_turn`` is the normal completion signal. Other stop reasons such as token limits or refusal should be handled explicitly rather than treated as success.

SIMPLE EXAMPLE

A purchase-order agent receives "show me the overdue POs for supplier A." Claude calls ``search_purchase_orders``. The application executes it, returns the records, and Claude then decides whether it has enough information to answer or needs another tool.

EXAM LENS

Prefer the API or SDK control signal over parsing natural-language text. An iteration cap is useful as a safety guardrail, but it should not be the primary definition of task completion.

COMMON TRAP

Stopping because the response contains a text block. Claude can return text and tool calls in the same response, so content type is not a safe completion test.

Remember it this way: Claude decides the next action. Your application owns the loop and the safety boundaries.

1.2 Multi-Agent Orchestration

Choose and implement orchestration patterns for multiple specialized agents.

What it means: Multi-agent architecture means breaking one larger goal into work owned by specialized agents, with an orchestration pattern that decides who delegates, who combines results, and how failures are handled.

How it works: A common exam pattern is hub-and-spoke: a coordinator owns the overall request, delegates focused tasks to specialists, receives their outputs, and synthesizes the final answer. Independent work can fan out in parallel; dependent work stays sequential. The coordinator should also inspect returned results for coverage. If a specialist exposes a gap, the coordinator can delegate a targeted follow-up before synthesis. That iterative delegate -> inspect -> delegate again pattern is more agentic than a one-shot fan-out pipeline. Central coordination improves observability and consistent error handling, but it adds cost and latency, so specialization should earn its complexity.

SIMPLE EXAMPLE

A research coordinator asks a financial-data agent for quantitative facts, a policy agent for regulations, and a document-analysis agent for source evidence. The policy result is missing the effective date, so the coordinator sends one focused follow-up to the policy agent before producing the final report.

EXAM LENS

Choose multi-agent design because the work is decomposable, parallel, context-heavy, or requires different tool permissions. If the returned work reveals a specific gap, a targeted second delegation is better than blindly synthesizing incomplete results.

COMMON TRAP

Treating orchestration as a fixed one-shot fan-out, or over-engineering a simple request with several agents when one agent already has the data and tools needed to finish it.

Remember it this way: Coordinate, inspect coverage, then delegate again only when the result creates a real need.

1.3 Subagent Invocation and Context Passing

Invoke subagents and pass enough context for them to succeed without bloating the parent context.

What it means: A subagent is a focused worker with its own context and tool permissions. In the exam-guide coordinator-subagent pattern, the coordinator must have the mechanism that can spawn the specialist, and the specialist does not automatically inherit the parent's conversation history.

How it works: Exam-guide framing: delegation uses the `Task` mechanism, so the coordinator's `allowedTools` must include `Task`. A specialist definition carries a name or description, its own instructions, and a restricted tool set so the coordinator can route work appropriately. Pass the goal, relevant facts, constraints, required output format, and success criteria explicitly in the task prompt. Give the specialist only the tools it needs. Independent tasks can be spawned in parallel when latency matters. The parent should receive a compact result rather than the specialist's entire intermediate exploration. Claude's product surfaces evolve, so current interfaces may use newer agent terminology, but for CCAR-F answer according to the published exam-guide mechanism.

SIMPLE EXAMPLE

A contract-review coordinator has `Task` available and delegates the liability section to a legal-analysis specialist. The task includes the relevant contract text, the date range, review criteria, required citations, and a structured risk format. A second independent section can be delegated in parallel.

EXAM LENS

If the prompt says 'delegate' but the coordinator keeps doing the work itself, check whether the spawning mechanism is actually allowed. If a spawned specialist gives a weak answer, check the context and output contract before changing the model.

COMMON TRAP

Assuming a subagent automatically knows facts from the parent conversation, forgetting to enable the spawning mechanism, or sending the entire parent transcript when only a small subset is relevant.

Remember it this way: Spawning is a mechanism. Context passing is a contract. You need both.

1.4 Workflow Enforcement and Handoff

Enforce mandatory workflow rules and create useful handoffs when work moves to another actor.

What it means: Prompts are good for guidance. Mandatory workflow invariants need a control outside the model, such as a hook, permission rule, state check, or application gate.

How it works: Before a sensitive tool executes, verify the prerequisite state. If the prerequisite is missing, block the tool and return a structured reason that helps Claude choose the correct next action. For human handoff, pass a structured packet with the identifiers, diagnosis, relevant amounts or evidence, attempted actions, and recommended next step. Do not make the human reconstruct the case from a raw transcript.

SIMPLE EXAMPLE A refund agent may process refunds up to \$500. For \$650, a pre-execution control blocks <code>`process_refund`</code> and directs the workflow to <code>`escalate_to_human`</code>, carrying customer ID, order ID, amount, policy reason, and prior steps.	EXAM LENS Words such as "must", "cannot", and "requires approval" are clues that a deterministic mechanism may be needed. Ask what actually guarantees the requirement.
---	--

COMMON TRAP Repeating the policy at the top and bottom of the system prompt and assuming repetition turns a probabilistic instruction into a guarantee.
--

Remember it this way: Prompt for behavior. Enforce invariants with code, permissions, or hooks.

1.5 Agent SDK Hooks

Use lifecycle hooks to inspect, block, modify, log, or react to agent behavior at defined control points.

What it means: Hooks are programmatic callbacks around agent lifecycle events. They let your application intervene at a known moment instead of hoping the model remembers an instruction.

How it works: Hooks run at defined lifecycle points. ``PreToolUse`` fires after Claude proposes a tool call but before the tool executes, so it can inspect inputs, allow, deny, request approval, or modify the proposed call. ``PostToolUse`` fires after a successful tool call, so it can validate, normalize, redact, log, or add context to the result Claude will reason over. A post-tool hook cannot undo an irreversible side effect that already happened. Other hook events exist, but the exam distinction to internalize is timing: prevent before execution; validate or normalize after execution.

SIMPLE EXAMPLE Before <code>`process_refund`</code> runs, a <code>`PreToolUse`</code> hook checks <code>`identity_verified`</code> and the amount and can block the call. Separately, a supplier-status tool returns dates and status codes in inconsistent formats; a <code>`PostToolUse`</code> hook normalizes them into one contract before Claude reasons over the result.	EXAM LENS Match the hook to the moment of control. Preventing an action belongs before execution. Normalizing or validating a successful result belongs after execution.
--	---

COMMON TRAP Using <code>`PostToolUse`</code> for a rule that must prevent an irreversible action, or using <code>`PreToolUse`</code> when the real problem is inconsistent output returned by a tool.
--

Remember it this way: Pre controls the action. Post controls what Claude receives after the action.

1.6 Task Decomposition Strategies

Break complex work into the right sequence of smaller tasks without losing global coherence.

What it means: Decomposition is about deciding whether the work is predictable enough for a fixed chain or open-ended enough to need adaptive planning.

How it works: For predictable multi-aspect work, prompt chaining can use a fixed sequence: analyze parts, then integrate. For open-ended investigation, use dynamic decomposition: map the problem, prioritize high-value areas, inspect findings, and revise the plan as new dependencies appear. These patterns can be combined. Splitting by file or section can also reduce attention dilution before a final synthesis pass.

SIMPLE EXAMPLE For a 20-file architecture review, first review each service against the same rubric, then run a cross-service synthesis to find systemic patterns. For an unfamiliar outage, start by mapping likely components, inspect the most probable causes, and adapt as evidence appears.	EXAM LENS Ask whether the next steps are knowable before execution. Fixed work favors a chain. Discovery work favors adaptive planning.
--	--

COMMON TRAP Choosing adaptive multi-agent planning for a mechanical change just because the task touches many files.

Remember it this way: The key axis is predictability and dependency, not task size.

1.7 Session State and Resumption

Continue, branch, or restart sessions while managing stale context and external state.

What it means: A session preserves conversation state. Resuming keeps the earlier reasoning context. Forking creates a new branch from a shared baseline so two approaches can diverge independently.

How it works: Use resume when the prior context is still useful. Use a fork when you want an independent branch without redoing the shared analysis. Remember that session history is not the same thing as the current filesystem, database, or business system. Old tool results can become stale, so re-read or re-query facts that may have changed. If the session is heavily contaminated with obsolete state, a new session plus a concise structured summary can be cleaner.

SIMPLE EXAMPLE Yesterday a coding agent analyzed a module. Today you want to compare two refactoring designs from that exact analysis. Fork the session into two branches. If the source files changed overnight, re-read the changed files in each branch before trusting the old findings.	EXAM LENS Separate "conversation continuity" from "external-state freshness." Resuming does not guarantee that earlier tool outputs are still current.
---	---

COMMON TRAP Treating a resumed session as if it automatically detects every change in the outside world.

Remember it this way: Resume context. Re-check state.

Tool Design & MCP Integration

WHAT THIS DOMAIN IS REALLY ASKING

Can Claude reliably discover the right capability, send valid inputs, interpret results and errors, and stay inside the right integration and permission boundary?

2.1 Tool Interface Design

Design tool interfaces that the model can select and call reliably.

What it means: A tool interface is the contract Claude sees: tool name, description, and input schema. Good backend code cannot compensate for an ambiguous contract.

How it works: Use a specific name, explain the purpose, describe the expected inputs and outputs, and state when to use this tool versus a similar one. In JSON Schema, ``properties`` are the fields Claude can send, and ``required`` lists the fields that must be present. Prefer purpose-specific interfaces over one giant generic tool with many loosely related modes.

SIMPLE EXAMPLE

``check_po_status`` takes required ``po_id: string`` and optional ``include_history: boolean``. Its description says it retrieves the current lifecycle status of one purchase order. A separate ``search_purchase_orders`` tool is for finding POs by supplier, date, or status.

EXAM LENS

When two tools are confused, look for overlapping names, vague descriptions, or missing intent boundaries before changing models or adding routing infrastructure.

COMMON TRAP

Adding more prompt keywords to compensate for two tools whose interfaces are genuinely ambiguous. The ambiguity is still present and may fail in the opposite direction.

Remember it this way: The description is part of the routing logic.

2.2 Structured Error Responses

Return errors that tell the agent what failed and what recovery is possible.

What it means: "Operation failed" is not enough. The agent needs to distinguish a temporary outage from invalid input, a policy rejection, or missing permission.

How it works: MCP tool results can use ``isError`` to signal tool-level failure. Beyond that protocol flag, your application can return structured details such as error category, human-readable message, retry guidance, and partial results. A valid empty result should usually remain a successful result, not be mislabeled as an error. Retry only when the failure is actually retryable.

SIMPLE EXAMPLE

A supplier API returns HTTP 503. The tool returns ``isError: true``, category ``transient``, and guidance that one bounded retry is allowed. A refund blocked by policy returns category ``business_policy`` and ``retryable: false``.

EXAM LENS

Look for the recovery path. The best error design helps the agent decide whether to retry, correct inputs, ask for access, choose another path, or escalate.

COMMON TRAP

Retrying every failure the same number of times. Repeated retries on policy or permission errors add latency and can create unsafe behavior.

Remember it this way: An error should help the next decision.

2.3 Tool Distribution and Tool Choice

Give each agent the right tool set and use tool-choice controls for the right purpose.

What it means: Tool distribution is a context and routing decision. An agent that sees many irrelevant or overlapping tools has a harder selection problem and a larger attack or error surface.

How it works: Scope tools to the agent's role and keep cross-role access intentional. Claude APIs support tool-choice modes such as ``auto`` (Claude may answer or call a tool), ``any`` (Claude must call a tool and chooses which one), and a specific named tool. These settings control tool invocation, not the ordering of an entire workflow. Modern Claude Code can also defer some MCP tool schemas until needed, reducing context pressure, but clear scoping still matters.

SIMPLE EXAMPLE

A billing agent gets account lookup, invoice lookup, and payment tools. It does not need code-editing or supplier-contract tools. For a structured extraction request where a tool call is mandatory, force tool use. For a normal support conversation, ``auto`` may be appropriate.

EXAM LENS

Ask what the control actually guarantees. ``tool_choice=any`` guarantees some tool call, not "verify identity before refund."

COMMON TRAP

Using tool choice as a substitute for workflow state or prerequisite checks.

Remember it this way: Tool selection and workflow enforcement are different problems.

2.4 MCP Server Integration

Connect Claude to external systems through MCP with the right scope, transport, credentials, and primitive.

What it means: MCP is a standard client-server protocol for exposing capabilities and context to AI applications. It reduces custom integration plumbing, but it does not remove the need for good tool design, authentication, or least privilege.

How it works: In Claude Code, project-scoped MCP configuration can live in ``.mcp.json`` and be shared through version control. Local and user scopes are private and stored in user configuration. Environment-variable expansion lets teams reference secrets without committing them. For remote servers, current Claude Code documentation favors HTTP; legacy SSE configurations may still exist. MCP defines three major server primitives: tools for model-invoked functions, resources for application-controlled context/data, and prompts for reusable user-controlled prompt templates.

SIMPLE EXAMPLE

A team adds a project-scoped Jira MCP server in ``.mcp.json``, references ``${JIRA_TOKEN}`` rather than committing the token, and exposes a narrow ``get_issue`` tool plus resources for read-only project metadata.

EXAM LENS

Choose scope based on who needs the server. Choose a tool when the model needs to invoke a function. Choose a resource when exposing readable context is the better fit. Prefer an existing trusted server when it meets the requirement; build custom behavior when the organization needs a specific contract or policy.

COMMON TRAP

Putting personal credentials directly into the repository, or creating a custom server for a standard integration without a real need.

Remember it this way: MCP standardizes the connection. Security and interface quality still belong to you.

2.5 Built-in Tools

Choose Claude Code built-in tools based on whether you are looking for content, paths, or a file operation.

What it means: The exam expects you to know the practical difference between common built-in tools. The choice should match the information you need, not the size of the repository.

How it works: ``Grep`` searches inside file contents for symbols, imports, error strings, or other text. ``Glob`` matches file paths and filename patterns. ``Read`` loads a file. ``Edit`` makes a targeted change. ``Write`` creates or replaces file content. ``Bash`` runs shell commands such as tests, builds, package commands, or Git operations. For unfamiliar repositories, search first, read selectively, and follow references instead of loading everything. If an ``Edit`` target is ambiguous, re-read the file and use a more unique anchor. If a precise edit is no longer practical and a whole-file rewrite is genuinely safer, use Read + Write rather than guessing at an ambiguous edit.

SIMPLE EXAMPLE

To find every caller of ``calculateTax``, use ``Grep``. To find every ``*.test.tsx`` path, use ``Glob``. To run ``npm test``, use ``Bash``. After locating the correct file, use ``Read`` and ``Edit``; if the edit anchor appears in six places, narrow the anchor before changing anything.

EXAM LENS

Translate the question into one of three categories: inside file contents, file paths, or an actual shell command. That usually points to Grep, Glob, or Bash. Then use Read/Edit/Write only on the files you have narrowed down.

COMMON TRAP

Bulk-reading hundreds of files for 'complete context', or reaching for Bash when a purpose-built search or file tool is safer and clearer.

Remember it this way: Search contents with Grep, paths with Glob, commands with Bash, then edit narrowly.

Claude Code Configuration & Workflows

WHAT THIS DOMAIN IS REALLY ASKING

Can a team make Claude Code behave consistently across people, repositories, file types, workflows, and automated pipelines?

3.1 CLAUDE.md Hierarchy, Scoping, and Modular Organization

Place persistent instructions at the right level so the right people and files receive them.

What it means: `CLAUDE.md` is persistent guidance for Claude Code. Its location controls scope. This is less about memorizing paths and more about asking "who should see this instruction?"

How it works: Typical scopes include user-level `~/claude/CLAUDE.md` for personal guidance, project-level `./CLAUDE.md` or `./.claude/CLAUDE.md` for team-shared instructions, and `CLAUDE.local.md` for private project-specific notes. Nested `CLAUDE.md` files can load when Claude works in those areas. `@import` can keep instruction sets modular, while `claude/rules/` supports topic-specific and path-specific organization. For the exam, know `/memory` as the direct command for listing and inspecting `CLAUDE.md`, local memory, and rules files. In current Claude Code, `/context` is a broader diagnostic that shows the live context breakdown and which memory files are loaded.

SIMPLE EXAMPLE

Your team has a testing standard that every developer should get. If it only lives in your home-directory `CLAUDE.md`, teammates will not inherit it. Move the shared standard to project scope and commit it. Use `/memory` to inspect the relevant memory/instruction files; use `/context` when you want the broader live context picture.

EXAM LENS

When the symptom is 'works for me but not for my teammate,' inspect configuration scope first. For CCAR-F questions centered on memory or `CLAUDE.md` loading, remember the exam-guide `/memory` framing; `/context` is the broader current-product diagnostic.

COMMON TRAP

Copying personal configuration by email to every teammate, or memorizing a command name without understanding that the real issue is configuration scope.

Remember it this way: First ask who should receive the instruction. Then put the file at that scope.

3.2 Custom Slash Commands and Skills

Package reusable, task-specific workflows so they load when needed rather than bloating always-on context.

What it means: Commands and skills are reusable instructions for a particular job. A good distinction is always-on project guidance versus an on-demand workflow.

How it works: Project commands and skills can be committed under `claude/`, while personal ones live in the user configuration area. A `SKILL.md` can include frontmatter that controls behavior. `context: fork` runs the skill in an isolated subagent context and returns its result, useful for verbose exploration. `allowed-tools` can restrict tools for Claude Code skill execution. `argument-hint` is an autocomplete hint that shows expected arguments; it is not a validation engine that magically asks for missing fields.

SIMPLE EXAMPLE A <code>/analyze-architecture</code> workflow scans many files and produces a concise risk report. Running it with <code>context: fork</code> keeps thousands of exploratory tokens out of the main session.	EXAM LENS Use CLAUDE.md for standards that should always influence work. Use a skill or command for a workflow invoked only when needed.
--	---

COMMON TRAP Putting every possible workflow into root CLAUDE.md, causing irrelevant instructions to occupy context on every task.
--

Remember it this way: Always-on rule -> CLAUDE.md. On-demand procedure -> skill.

3.3 Path-Specific Rules for Conditional Convention Loading

Load conventions only when the files they apply to are relevant.

What it means: Path-specific rules prevent every coding convention from appearing in every context. They are especially useful when one file type is scattered across many directories.

How it works: A file in `.claude/rules/` can use `paths:` frontmatter with glob patterns. The rule is activated when Claude reads or works with matching files, keeping unrelated conventions out of context. A directory-level CLAUDE.md can also be appropriate when the convention naturally maps to one subtree. Choose by the shape of the repository.

SIMPLE EXAMPLE All <code>*.test.tsx</code> files across 15 directories must follow the same mocking convention. One rule with <code>paths: ["**/*.test.tsx"]</code> is cleaner than adding a CLAUDE.md to every directory.	EXAM LENS If the rule is based on file type across the repository, a glob is often the natural fit. If the rule belongs to one subsystem, directory scope may be simpler.
---	--

COMMON TRAP Making a universal root rule for a narrow convention and paying the context cost on every unrelated task.
--

Remember it this way: Scope instructions to where they are useful.

3.4 Plan Mode vs Direct Execution

Decide when Claude should analyze and design before changing files.

What it means: Plan mode is for exploration and design when the right implementation is not yet obvious or the cost of a wrong change is high. Direct execution is appropriate when the transformation is clear and mechanical.

How it works: In plan mode, Claude can inspect and reason without making source edits, then present a plan for approval. The decision is driven by ambiguity, architectural choice, risk, and rework cost, not simply the number of files. A large but deterministic rename can be executed directly; a two-file authentication redesign may deserve planning.

SIMPLE EXAMPLE Renaming one deprecated function in 60 files with one obvious replacement is mechanical. Reworking an authentication boundary with three valid approaches should be planned first, even if only a few files are involved.	EXAM LENS Read the condition behind the scary number. "60 files" does not automatically mean plan mode.
---	--

COMMON TRAP Using file count as the only trigger for plan mode.
--

Remember it this way: Plan for ambiguity, not for size.

3.5 Iterative Refinement Techniques

Give Claude objective feedback so each iteration is driven by evidence rather than vague preference.

What it means: Refinement works best when the system can compare an output with something concrete: tests, examples, schemas, expected behavior, or explicit acceptance criteria.

How it works: Provide representative input/output examples when prose instructions are being interpreted inconsistently. For code, give Claude a verification loop: reproduce the bug, write or run tests, make a focused change, and re-run the checks. In unfamiliar domains, an interview pattern can surface missing design decisions before implementation. Keep independent changes separable so one failure does not obscure another.

SIMPLE EXAMPLE A code-review agent keeps missing a specific API-contract violation. Add a failing test or a concrete positive/negative example, then iterate against that evidence rather than telling it to "be more careful."	EXAM LENS Prefer a feedback signal that can tell you exactly what failed. This turns iteration into debugging rather than prompt polishing.
--	--

COMMON TRAP Adding more and more vague instructions without a test, example, or validation target.

Remember it this way: A verification mechanism is better than another adjective.

3.6 CI/CD Integration

Run Claude Code non-interactively, provide deterministic inputs, and return outputs that automation can parse.

What it means: CI/CD has no person sitting at the terminal. The invocation must not wait for interactive input, and downstream systems need a predictable result format.

How it works: Use ``-p`` or ``--print`` for non-interactive execution. Use ``--output-format json`` and, when appropriate, ``--json-schema`` for machine-parseable structured output. Make repository standards, existing tests, fixtures, and review criteria available so Claude does not invent duplicate checks or miss project-specific expectations. Separate generation and review when independent review is required. On a repeated CI review, pass the prior findings or a clear baseline so the next pass can focus on new or unresolved issues instead of reporting the same items again. Pass only required credentials and permissions, and fail safely.

SIMPLE EXAMPLE A pull-request pipeline runs Claude Code to review code. It uses <code>`-p`</code>, outputs findings as JSON against a schema, and supplies the repository's existing tests and review rubric. A later rerun includes the previous findings so the pipeline can distinguish new issues from already-known ones.	EXAM LENS Hanging headless job -> check interactive mode. Brittle parsing -> structured output. Duplicate test suggestions -> provide existing tests. Repeated review noise -> provide the previous findings or baseline. Overly forgiving self-review -> use an independent review instance.
---	---

COMMON TRAP Increasing <code>`max_tokens`</code> to fix a job waiting for input, asking the same generation session to be its own independent reviewer, or rerunning a review with no memory of previously reported findings.
--

Remember it this way: CI needs non-interactive execution, explicit context, structured output, and an independent verification path when the risk demands it.

Prompt Engineering & Structured Output

WHAT THIS DOMAIN IS REALLY ASKING

Can you turn business intent into explicit criteria, produce predictable structured data, validate it, and choose the right processing and review pattern?

4.1 System Prompts with Explicit Criteria

Translate vague quality goals into observable decision criteria.

What it means: A model cannot reliably optimize a rule that has not been defined. "Be conservative" or "use good judgment" sounds responsible but does not tell Claude what should count as an error.

How it works: Write categorical criteria that the model can apply to evidence. Define what qualifies, what does not, how to handle ambiguous cases, and what output is expected. For precision problems, tighten the category definition before using arbitrary confidence thresholds. If one noisy category is hurting trust, it can be better to temporarily disable that category while you fix its definition.

SIMPLE EXAMPLE

A code-review agent should flag a comment only when the comment claims behavior that contradicts the actual code. That criterion is testable. "Flag comments only when you are very confident" is not.

EXAM LENS

Look for the option that defines the condition, not the option that asks Claude to feel more confident about an undefined condition.

COMMON TRAP

Using a 1-10 self-confidence score as a substitute for defining what a true positive is.

Remember it this way: Define the decision before asking the model to make it.

4.2 Few-Shot Prompting

Use targeted examples to teach the boundary of a task and the desired output pattern.

What it means: Few-shot prompting means showing the model representative examples of correct behavior. The value comes from choosing examples that clarify the decision boundary, not from adding as many examples as possible.

How it works: Include examples that represent common cases, difficult edge cases, and at least one negative or missing-information case when that matters. For ambiguous decisions, examples can show why one action is chosen over another. Keep examples consistent with the exact output format you want. If a source field may be absent, show an example where the correct output is `null` rather than an invented value.

SIMPLE EXAMPLE

For invoice extraction, provide one normal invoice, one invoice with a missing purchase-order number, and one with unusual line-item formatting. The missing field example teaches the model that absence is valid.

EXAM LENS

When prose is technically correct but results remain inconsistent, a small set of well-chosen examples can clarify the boundary more effectively than another paragraph of abstract instructions.

COMMON TRAP

Adding many near-duplicate examples that consume context without teaching new distinctions.

Remember it this way: Examples should teach the edge, not just the average.

4.3 Structured Output with Tool Use

Produce machine-readable data that conforms to a defined schema.

What it means: Structured output separates the model's reasoning ability from the downstream system's need for predictable fields and types. In the certification objective, tool use is a key mechanism for doing this.

How it works: A tool definition contains a name, description, and JSON input schema. For extraction, Claude can populate the tool input and your application can keep the `tool\_use` data rather than executing a business action. Tool choice can force a call when structured data is required. Current Claude APIs also support structured output features such as JSON output formats and strict tool use; `strict: true` can guarantee schema-conformant tool inputs. None of these guarantees that the business facts are correct. Make potentially absent fields optional or nullable. Use enums for fixed value sets, but give ambiguous real-world inputs an honest escape path such as `unclear` or `other` plus a detail field rather than forcing the wrong category.

SIMPLE EXAMPLE

A document extractor returns `{customer\_id, order\_id, issue\_type, issue\_type\_detail}`. If `order\_id` can be absent, it is nullable. If `issue\_type` is an enum, include `other` or `unclear` so an unexpected document is represented honestly instead of being forced into the closest wrong label.

EXAM LENS

Separate three questions: Is the output parseable? Does it match the schema? Is the content semantically correct? Then ask a fourth: can the schema honestly represent absence or ambiguity?

COMMON TRAP

Marking every field required, or defining an enum with no escape value even though the source can contain cases outside the known categories.

Remember it this way: Schema should constrain the answer without forcing the model to invent certainty.

4.4 Validation, Retry, and Feedback Loops

Validate structured outputs and retry only when the failure is recoverable.

What it means: Validation is the layer that checks whether an answer is actually usable. Retries become useful when they carry specific evidence about what failed.

How it works: Use schema validation for types and required fields, then add semantic or business validation such as totals, ranges, cross-field consistency, and policy constraints. On a recoverable failure, send the original source, failed output, and exact validation error so Claude can correct the specific issue. Bound retries. If the source does not contain the information, repeated retries encourage fabrication; change the schema or escalate instead. For review systems, preserve a category or `detected\_pattern` with each finding so dismissed false positives can be grouped later. That feedback lets you improve the actual decision criteria rather than repeatedly adding vague prompt text.

SIMPLE EXAMPLE An extracted invoice has valid JSON but line items sum to \$970 while <code>`stated_total`</code> is \$1,020. Validation computes <code>`calculated_total`</code>, sets <code>`conflict_detected`</code>, and triggers a targeted retry or review. In a code-review pipeline, rejected findings retain <code>`detected_pattern`</code> so you can see which rule is generating most false positives.	EXAM LENS A real feedback loop changes what the next attempt knows. Validator errors improve the immediate retry; categorized false positives improve the prompt or rule over time.
--	--

COMMON TRAP Retrying the identical prompt, retrying until a genuinely absent field becomes non-null, or collecting 'false positive' feedback without enough structure to learn which pattern is failing.

Remember it this way: Validate now, preserve the failure pattern, then improve the system with evidence.

4.5 Batch Processing Strategies

Choose asynchronous batching when throughput and cost matter more than immediate latency.

What it means: Batch processing is for large sets of independent requests that do not need an immediate response. It is an economic and operational design choice.

How it works: Anthropic's Message Batches API charges 50% of standard API pricing and is designed for large, independent requests that do not need an immediate answer. Most batches may finish sooner, but the processing window can extend to 24 hours, so do not treat it as a low-latency SLA. Results can arrive out of submission order, so every request needs a stable ``custom_id``. Test or refine the prompt on a representative sample before submitting a large job. Chunk work when needed and resubmit only failed requests rather than paying to rerun successful ones. For the exam, do not choose Batch for a workflow that requires interactive, multi-turn client-tool execution between model calls; use synchronous calls when a user or pipeline is blocked on the next turn.

SIMPLE EXAMPLE A nightly job extracts fields from 50,000 archived invoices. The team validates the prompt on a sample, submits the rest as a batch with unique <code>`custom_id`</code> values, and resubmits only failed documents. A pre-merge review that must call tools and continue immediately stays synchronous.	EXAM LENS Look for independent + non-blocking + latency-tolerant work. The exam numbers worth remembering are 50% pricing and up to 24 hours, plus <code>`custom_id`</code> because results can be out of order.
---	---

COMMON TRAP Using Batch for an interactive support request, assuming results return in submission order, or rerunning an entire batch because a small subset failed.

Remember it this way: Batch saves 50% when you can wait. ``custom_id`` keeps the asynchronous results straight.

4.6 Multi-Instance and Multi-Pass Review

Separate generation from review when an independent perspective can reduce anchoring and catch different failure modes.

What it means: Review is not the same task as generation. A separate pass can use a different rubric, context, or even a fresh instance so the reviewer is less anchored to the reasoning that produced the first answer.

How it works: A useful pattern is generate -> independent review -> revise. For a large multi-file review, split the review into focused per-file local passes, then run a separate cross-file integration pass that checks data flow, consistency, and contradictions across the collected findings. A fresh reviewer should receive the artifact, standards, tests, and review rubric, not the generator's reasoning context. A verification pass may attach a self-rated confidence to each finding, but use that

score for automated routing only after calibrating it against labeled outcomes. Self-review is still useful for fast iteration; it is not the same as independent review.

SIMPLE EXAMPLE A 14-file pull request gets inconsistent depth in a single review. The system reviews each file independently, then gives all findings to a separate integration reviewer for cross-file issues. A fresh verification instance scores findings, and the team uses labeled review history to calibrate which confidence range actually needs human review.	EXAM LENS Too-forgiving review -> fresh instance. Attention dilution across many files -> per-file passes plus cross-file integration. Confidence-based routing -> calibrate the score against real outcomes before trusting the threshold.
---	---

COMMON TRAP Using one huge review pass, asking the generator to 'be more critical' in the same context, or treating an uncalibrated self-confidence number as a probability of correctness.
--

Remember it this way: Fresh reviewer, focused passes, integration pass, calibrated routing.

Context Management & Reliability

WHAT THIS DOMAIN IS REALLY ASKING

Can the system remain grounded, recover from failure, handle ambiguity, preserve exact facts and sources, and know when a human should take over?

5.1 Context Window Management

Keep the most decision-relevant information available as conversations and tool outputs grow.

What it means: Context is the working set Claude can attend to in the current request. More context is not automatically better context. Long sessions can accumulate repeated instructions, verbose tool outputs, and old facts that compete with what matters now.

How it works: Protect exact facts such as IDs, dates, amounts, and user requirements in a compact structured note rather than allowing them to be repeatedly paraphrased. Trim large tool outputs to relevant fields. Use clear sectioning and place critical constraints where they are easy to find. Claude Code can compact long sessions, and `/compact` can be used explicitly. Subagents can isolate verbose exploration. A larger context window can be useful, but it does not fix poor information architecture by itself.

SIMPLE EXAMPLE

A support case has a \$247.83 disputed charge, order 8421, and a promised date of August 30. Keep those exact facts in a small case-facts block while older conversational detail is summarized.

EXAM LENS

When quality drops late in a session, ask whether the problem is missing information or attention dilution. The fixes are different.

COMMON TRAP

Automatically switching to a bigger context window without trimming irrelevant output or protecting exact facts.

Remember it this way: The goal is not to hold everything. It is to keep what the next decision needs.

5.2 Escalation and Ambiguity Resolution

Resolve uncertainty without guessing, and escalate when the scenario policy requires it.

What it means: Reliable agents need explicit behavior for cases they cannot safely resolve. The correct action depends on the stated policy, not on a generic notion of confidence or customer sentiment.

How it works: If the customer explicitly requests a human and the scenario policy says to honor that request, escalate immediately. If multiple records could match, ask for another identifier rather than choosing heuristically. If the policy has a real gap or an exception outside the agent's authority, route it to the appropriate human or policy owner. Frustration alone is not always the same as an explicit escalation request.

SIMPLE EXAMPLE

Two customers share the same name and ZIP code. The agent should ask for an order number or another allowed identifier instead of selecting the first record. A customer saying "I want a human" should be handled according to the scenario's explicit escalation rule.

EXAM LENS

Look for real triggers defined by the scenario: explicit request, policy exception, or inability to proceed. Treat sentiment and self-reported model confidence as weak proxies unless the scenario explicitly defines them as policy.

COMMON TRAP

Guessing the most likely customer record because it has a slightly higher similarity score.

Remember it this way: When identity or policy is ambiguous, get evidence or escalate. Do not guess.

5.3 Error Propagation in Multi-Agent Systems

Let local components recover when they can, but preserve enough error context when failure must move upstream.

What it means: A multi-agent system should neither hide every failure nor collapse the entire workflow whenever one specialist has a problem.

How it works: Handle retryable transient errors close to where they occur, with limits. When an error cannot be resolved locally, propagate structured context: what failed, the category, what was already attempted, any partial result, and reasonable alternatives. The coordinator can then decide whether to continue with partial coverage, call another source, ask the user, or escalate. Preserve which subtask failed so the final output does not overstate completeness.

SIMPLE EXAMPLE

Three research subagents run in parallel. One source times out twice. The subagent returns the two successful partial facts from its work plus `source\_unavailable`, attempts made, and the missing coverage. The coordinator can finish with an explicit gap or choose another source.

EXAM LENS

Good error propagation preserves options. It should give the coordinator enough information to make a deliberate next decision.

COMMON TRAP

Silently dropping the failed subagent and writing a report that appears complete, or terminating all other successful work because one source failed.

Remember it this way: Recover locally when possible. Propagate facts, not just failure.

5.4 Codebase Exploration and Context Degradation

Explore unfamiliar repositories without flooding the main context with low-value detail.

What it means: Codebase exploration is an information-retrieval problem. The goal is to find the smallest set of files and relationships needed to understand the behavior in question.

How it works: Start with search: Grep for symbols, error strings, or entry points; Glob for relevant path patterns. Read only the files that surface, then follow imports and call relationships. Delegate especially verbose investigation to a subagent that returns a compact summary. Maintain a scratchpad or findings file for exact discoveries that must survive long sessions. If the main context becomes noisy, compact or start a cleaner branch with a structured summary.

SIMPLE EXAMPLE

To understand authentication in an 800-file service, search for the auth middleware and token validation entry points, read those files, then follow imports. Do not read all 800 files "for completeness."

EXAM LENS

When Claude starts answering from generic patterns rather than the specific code it inspected, you probably have a context-quality problem, not simply an intelligence problem.

COMMON TRAP

Bulk-reading the repository or repeatedly asking the model to "remember the code better" while continuing to add more output.

Remember it this way: Navigate the codebase. Do not pour it into the context window.

5.5 Human Review and Confidence Calibration

Route work to humans based on measured reliability and risk, not unvalidated self-confidence.

What it means: Human review is most useful when it is targeted. A model saying "I am 92% confident" is not automatically a calibrated probability of correctness.

How it works: Measure model performance against labeled examples and inspect accuracy by important segment, such as document type, field, language, or risk category. Aggregate accuracy can hide a weak minority segment. Use stratified sampling and ongoing review to estimate real error rates. Route ambiguous, high-risk, policy-sensitive, or empirically low-performing cases to humans. If a review or verification pass emits a confidence score, calibrate that score against observed outcomes before using it as a routing threshold. The number becomes useful only after you know what it means on real labeled cases.

SIMPLE EXAMPLE	EXAM LENS
An extractor is 96% accurate overall but only 78% on handwritten invoices. Do not automate all documents based on 96%. Route the handwritten segment for review while improving that path.	Look for evidence-backed routing: measured segment performance, ambiguity, or business risk. Be skeptical of raw self-reported confidence.

COMMON TRAP
Choosing a human-review threshold of "confidence < 0.8" without ever checking whether 0.8 has any relationship to correctness.

Remember it this way: Calibration comes from measured outcomes, not from a number the model invents.

5.6 Information Provenance and Multi-Source Synthesis

Preserve where facts came from as information moves through research and synthesis.

What it means: A useful answer is not fully trustworthy if nobody can trace its claims back to sources. Provenance must survive summarization, delegation, and synthesis.

How it works: Have research agents return structured claim-source mappings that carry the claim, source or document identifier, relevant evidence, and publication or observation date when timing matters. The synthesis layer should preserve those mappings instead of flattening everything into unattributed prose. When credible sources conflict, retain both values with attribution and investigate date, scope, or methodology differences. Do not silently average or choose one unless the business rule tells you how to reconcile them.

SIMPLE EXAMPLE	EXAM LENS
One source reports market adoption at 41% in March and another reports 58% in July. The research system carries both values, sources, and dates. The final report explains that the numbers may reflect different measurement periods rather than pretending one is "the truth."	If a question says citations disappeared after summarization, the structural fix is to preserve attribution in the intermediate data contract, not to ask the final agent to guess sources afterward.

COMMON TRAP
Keeping a generic list of "sources consulted" while losing the mapping between each claim and the source that supports it.

Remember it this way: Every important claim should be able to answer: where did this come from?

SECTION 7

Seven patterns that resolve many difficult questions

These are exam heuristics, not universal laws. Use them only when the scenario conditions support them.

1. Structural control beats prompt-only control when the requirement is a guarantee

If an action must never occur without a prerequisite, a prompt can help behavior but a gate, permission, hook, or workflow state is what enforces the invariant.

2. Ask what actually changed

If routing breaks immediately after a prompt change while tool interfaces did not change, inspect the new prompt before redesigning the tool layer. If two tool descriptions were always vague, fix the tool contracts.

3. The "sounds big" trap

A 60-file mechanical rename can be simpler than a two-file architectural redesign. Use ambiguity and rework cost to decide planning, not the size words in the stem.

4. Schema guarantees shape, not business truth

Valid JSON can contain an invalid invoice total. Use schemas for structure and a separate validation layer for semantics.

5. A larger context window is not a universal reliability fix

Use more context when missing information is the real problem. Use trimming, structure, compaction, subagents, or persistent exact facts when attention dilution is the problem.

6. Independent review is about a different evaluation context

A fresh review pass can reduce anchoring, especially when it receives the artifact plus a rubric rather than the generator's reasoning. It is not magic, and it does not replace tests or validation.

7. Run the mechanism check

Before choosing an answer, ask: does this option contain the mechanism that produces the stated result, or does it only restate the problem in better words?

ONE SENTENCE TO REMEMBER

Read the constraint, identify the failure mode, then pick the smallest mechanism that actually changes the outcome.

SECTION 8

25 original scenario-based practice questions

Attempt these closed-book. The questions are written against the public blueprint objectives and do not reproduce live exam content.

HOW TO USE THE QUESTIONS

Give yourself about 45-60 minutes for all 25. Mark the objective next to any question you were unsure about, even if you guessed correctly. The answer key explains the mechanism and the most tempting distractor.

Q1. Domain 1 | Select ONE

Customer Support Resolution Agent

The agent loop sometimes exits after Claude writes a short explanation and requests a tool in the same response. The application currently stops whenever it sees any text block. What is the best fix?

- A. Increase the maximum loop iterations from 5 to 20
- B. Use `stop\_reason` to control the loop, continuing on `tool\_use` and treating `end\_turn` as normal completion
- C. Tell Claude never to write text before a tool call
- D. Parse the last sentence for the word "done"

Q2. Domain 1 | Select ONE

Customer Support Resolution Agent

Refunds above \$500 require human approval. Prompt instructions occasionally allow a \$650 refund tool call. Which design best enforces the policy?

- A. Repeat the \$500 rule at the start of every turn
- B. Lower temperature so Claude is more consistent
- C. Add a `PreToolUse` hook or application gate that blocks `process\_refund` above \$500 and routes to approval
- D. Put the rule only in the refund tool description

Q3. Domain 2 | Select ONE

Customer Support Resolution Agent

`process\_refund` returns only "Operation failed" for every failure. The agent retries policy rejections but gives up immediately on temporary outages. What should change?

- A. Return structured error information such as category, retry guidance, and description
- B. Retry every error exactly twice
- C. Hide errors from Claude and let the application decide silently
- D. Ask Claude to apologize before every retry

Q4. Domain 5 | Select ONE

Customer Support Resolution Agent

The first customer message says, "I want a human. Do not try to resolve this." The scenario policy says explicit human requests must be honored. What should the agent do?

- A. Run sentiment analysis first
- B. Attempt one automated resolution, then escalate if it fails
- C. Escalate immediately with a structured handoff
- D. Ask three diagnostic questions before deciding

Q5. Domain 1 | Select ONE

Customer Support Resolution Agent

One customer message contains a damaged-item refund, an address update, and a missing loyalty credit. The subtasks are known and can be handled separately before one combined response. Which design best fits?

- A. Use a fixed decomposition into the three concerns, resolve each, then synthesize
- B. Escalate because multi-issue messages require humans
- C. Build an adaptive planner that can invent new subtasks indefinitely
- D. Resolve only the first concern and ignore the rest

Q6. Domain 2 | Select TWO

Customer Support Resolution Agent

The agent often confuses ``get_customer`` and ``lookup_order``. Both have one-line descriptions that mention "customer order data." Which TWO changes most directly improve routing?

- A. Add expected input format and output details to each tool description
- B. State when each tool should be used instead of the other
- C. Increase the context window
- D. Give both tools shorter names with the same prefix
- E. Add a generic "choose carefully" instruction to the system prompt

Q7. Domain 3 | Select ONE

Code Generation with Claude Code

Team coding standards live only in ``~/ .claude/CLAUDE.md``, so a new teammate does not receive them. What is the best fix and exam-focused verification step?

- A. Move shared standards to project-level CLAUDE.md, commit them, and use ``/memory`` to inspect the relevant memory/instruction files
- B. Email the teammate your home-directory file
- C. Reinstall Claude Code on the teammate's machine
- D. Paste the standards manually at the start of every session

Q8. Domain 3 | Select ONE

Code Generation with Claude Code

A reusable ``/analyze-architecture`` skill generates thousands of lines of exploration and crowds the main conversation. Which skill design best isolates that work?

- A. Add ``argument-hint``
- B. Use ``context: fork`` so the skill runs in an isolated context and returns the result
- C. Move the workflow into root CLAUDE.md
- D. Increase max output tokens

Q9. Domain 1 | Select ONE

Code Generation with Claude Code

You finished a baseline module analysis and now want two independent refactoring approaches from that exact starting point. What is the best session strategy?

- A. Ask the same session to forget the first approach
- B. Fork the session into separate branches from the baseline
- C. Start two completely fresh sessions and redo all analysis
- D. Use ``/compact`` between the two approaches

Q10. Domain 3 | Select ONE

Code Generation with Claude Code

A deprecated function must be renamed across 60 files. The replacement is mechanical and there is one obvious implementation. Plan mode or direct execution?

- A. Plan mode because more than 10 files are involved
- B. Plan mode because any repository-wide change is architectural
- C. Direct execution because the change is unambiguous and low rework-risk
- D. Spawn multiple research subagents because the file count is high

Q11. Domain 5 | Select TWO

Developer Productivity with Claude

After hours of exploration, the agent starts answering from generic patterns instead of the exact classes it inspected. Which TWO actions best address the context-quality problem?

- A. Keep a compact scratchpad of exact findings and important file references
- B. Delegate verbose exploration to subagents that return concise findings
- C. Keep loading more files so everything remains in one context
- D. Repeat each question twice
- E. Tell Claude to be more confident

Q12. Domain 1 | Select ONE

Multi-Agent Research System

The coordinator is instructed to delegate to a specialist, but it keeps doing all the work itself. The specialist is defined correctly, but the coordinator does not have `Task` in its allowed tool set. What is the most direct fix?

- A. Add `Task` to the coordinator's allowed tools and pass the specialist a complete task brief
- B. Increase the coordinator model size
- C. Copy the specialist's whole prompt into the coordinator
- D. Start the specialist manually before every user request

Q13. Domain 1 | Select ONE

Multi-Agent Research System

Three specialists run in parallel. The policy specialist returns useful findings but says the effective date is missing. What should the coordinator do before synthesis?

- A. Ignore the gap because the other two specialists succeeded
- B. Send a targeted follow-up delegation for the missing date, then synthesize when coverage is sufficient
- C. Restart all three specialists from scratch
- D. Average the available dates from unrelated sources

Q14. Domain 5 | Select ONE

Multi-Agent Research System

The final report can no longer identify which source supports each claim because attribution was lost during summarization. What is the strongest structural fix?

- A. Ask the final writer to guess likely sources from memory
- B. Preserve structured claim-source mappings through each subagent and synthesis step
- C. Add a generic "sources consulted" list at the end
- D. Use a larger context window and keep everything as prose

Q15. Domain 5 | Select ONE

Multi-Agent Research System

Three research subagents run in parallel. One source times out after bounded retries, while the other two succeed. What should the failed subagent return upstream?

- A. An empty success result so the coordinator can continue quietly
- B. A generic "search unavailable" string with no other detail
- C. Structured failure context including what failed, attempts made, partial results, and the missing coverage
- D. A signal that terminates the entire workflow immediately

Q16. Domain 2 | Select TWO

Developer Productivity with Claude

Which TWO built-in-tool mappings are correct?

- A. Find every caller of `calculateTax` -> `Grep`
- B. Find all `*.test.tsx` file paths -> `Grep`
- C. Run `npm test` -> `Bash`
- D. Load one known source file -> `Glob`
- E. Make a targeted text replacement -> `Bash`

Q17. Domain 2 | Select ONE

Structured Data Extraction

A document may match one of several extraction tools, and the system requires Claude to call some extraction tool on every request while allowing Claude to choose which schema fits. Which `tool_choice` behavior best matches?

- A. `auto`
- B. `any`
- C. A fixed specific tool for every document type
- D. No tools; parse natural-language JSON with regex

Q18. Domain 2 | Select ONE

Developer Productivity with Claude

Agents waste calls discovering what reference documents exist before they can work. The server can expose a browsable catalog of those documents. Which MCP primitive is the natural fit?

- A. A resource that exposes the catalog
- B. A destructive write tool
- C. A second system prompt
- D. An iteration limit

Q19. Domain 3 | Select ONE

Developer Productivity with Claude

A testing convention applies to all `*.test.tsx` files scattered across 15 directories and should not load for normal source files. What is the best mechanism?

- A. A `.claude/rules/` file with a matching `paths` glob
- B. Put it in root `CLAUDE.md` so it is always loaded
- C. Create 15 user-level `CLAUDE.md` files
- D. Rely on developers to paste the rule manually

Q20. Domain 3 | Select TWO

Claude Code for CI/CD

A CI job invokes Claude Code for PR review. It sometimes hangs waiting for input, and the pipeline also needs findings it can parse automatically. Which TWO changes are most direct?

- A. Add ``-p` / `--print`` for non-interactive execution
- B. Use ``--output-format json`` with an appropriate ``--json-schema``
- C. Increase ``max_tokens``
- D. Parse free-form prose with regular expressions
- E. Ask a human to watch every CI run

Q21. Domain 3 | Select ONE

Code Generation with Claude Code

Claude repeatedly applies an inconsistent fix for the same API-contract bug. Which refinement step gives the strongest objective feedback?

- A. Add "be more careful" to CLAUDE.md
- B. Create or provide a failing test that captures the bug, make a focused change, and rerun the test
- C. Increase temperature so Claude explores more alternatives
- D. Add several paragraphs of general coding advice

Q22. Domain 1 | Select ONE

Customer Support Resolution Agent

A supplier-status tool succeeds, but it returns dates and status codes in inconsistent formats. Claude then misinterprets some results. Which hook placement best addresses this?

- A. ``PreToolUse`` to block the tool before it runs
- B. ``PostToolUse`` to normalize the successful tool output before Claude reasons over it
- C. A system prompt saying "interpret formats carefully"
- D. A session fork after every tool call

Q23. Domain 4 | Select TWO

Structured Data Extraction

An extraction system has two failures: ``middle_name`` is required even when absent, causing invented initials; and invoice line items sometimes do not sum to the stated total even though the JSON matches the schema. Which TWO changes address the root causes?

- A. Make ``middle_name`` optional or nullable
- B. Add semantic validation such as ``calculated_total`` versus ``stated_total``
- C. Retry until every document contains a middle name
- D. Remove all schema constraints
- E. Convert all numbers to strings

Q24. Domains 4 & 5 | Select THREE

Claude Code for CI/CD

A 14-file pull request gets inconsistent depth in a single review. The same generation session is too forgiving when asked to review its own code, and the team wants to route uncertain findings to humans. Which THREE changes form the strongest review architecture?

- A. Review files in focused local passes, then run a separate cross-file integration pass
- B. Use a fresh review instance that receives the artifact and review rubric without the generator's reasoning context
- C. Calibrate any confidence-based routing threshold against labeled outcomes before trusting it
- D. Keep one giant review pass but increase the context window
- E. Ask the generating session to "think harder" and trust any confidence score above 0.8

Q25. Domain 4 | Select TWO**Structured Data Extraction**

A nightly job will process 50,000 independent archived documents. Results are not needed until tomorrow morning. Which TWO design choices fit best?

- A. Use the Message Batches API, which is priced at 50% of standard API rates for this latency-tolerant workload
- B. Give each request a stable `custom\_id` so out-of-order results can be correlated
- C. Keep a user waiting in an interactive session for every document
- D. Assume every result returns in submission order and omit identifiers
- E. Convert the batch into one giant multi-turn client-tool conversation

SECTION 9

Answer key with rationales

Do not stop at the letter. Read the rationale and identify the mechanism that makes the answer correct.

Q1 - B

The bug is using text content as a completion signal. A response may contain both text and a tool request. Drive the loop from structured control state: continue when client tool use requires another round trip, treat `end\_turn` as normal completion, and handle other stop reasons explicitly.

Q2 - C

The requirement is a hard business invariant. A pre-execution hook or application gate can block the refund before the side effect occurs. Prompt repetition or lower temperature may improve behavior but does not create the same enforcement mechanism.

Q3 - A

The next action depends on the failure type. Structured error context lets the agent distinguish transient, validation, policy, and permission failures and decide whether to retry, correct input, ask for access, or stop.

Q4 - C

The scenario provides a direct escalation trigger. Honor the explicit human request immediately and send a useful structured handoff rather than forcing another automated attempt.

Q5 - A

The concerns are known in advance and can be handled as a predictable chain. Fixed decomposition is easier to control and test than an adaptive planner when no discovery is required.

Q6 - A and B

Tool selection depends on the visible contract. Input/output details reduce ambiguity, and an explicit boundary saying when to use one tool instead of the other distinguishes intent. More context or generic caution does not repair underspecified interfaces.

Q7 - A

The root cause is scope. Shared standards belong in project-level CLAUDE.md so version control distributes them. The exam-guide memory workflow emphasizes `/memory` for inspecting the relevant CLAUDE.md and memory files; current `/context` is useful for a broader live context view.

Q8 - B

`context: fork` isolates verbose workflow execution and returns only the result to the parent context. `argument-hint` helps users see expected arguments; it does not isolate context.

Q9 - B

Forking preserves the common baseline while creating independent branches for divergent approaches. `/compact` is context management, not branching, and fresh sessions discard useful shared analysis.

Q10 - C

The deciding conditions are ambiguity, risk, and rework cost, not file count. A mechanical rename with one obvious path is proportionate for direct execution.

Q11 - A and B

A compact findings record preserves exact discoveries, while subagents isolate verbose exploration. Loading more files adds to the same context-quality problem.

Q12 - A

Prompt instructions cannot spawn a specialist if the spawning mechanism is unavailable. In the exam-guide pattern the coordinator needs `Task` in its allowed tools, and the task itself still needs the relevant goal, facts, constraints, and output contract.

Q13 - B

A coordinator should inspect specialist outputs before synthesis. When a returned result exposes a specific gap, a targeted second delegation is cheaper and more reliable than restarting everything or hiding the missing coverage.

Q14 - B

Provenance must survive intermediate handoffs. Claim-source mappings preserve attribution through summarization and synthesis; a generic bibliography does not tell you which source supports which claim.

Q15 - C

Good error propagation preserves recovery options. The coordinator needs failure type, attempts, partial work, and missing coverage so it can choose another source, continue with an explicit gap, or escalate. Empty success and workflow-wide termination are opposite anti-patterns.

Q16 - A and C

`Grep` searches inside file contents, so it is the right tool for callers of a symbol. `Bash` runs shell commands such as `npm test`. `Glob` is for paths, `Read` for known files, and `Edit` for targeted changes.

Q17 - B

`tool\_choice=any` requires some tool call while letting Claude choose which available tool fits. `auto` allows a text-only response, and forcing one specific tool would be wrong when the document type determines the schema.

Q18 - A

The need is discoverability of readable context. An MCP resource is a natural fit for a browsable catalog, while a tool is better when the model needs to invoke an action.

Q19 - A

The convention is conditional on file type and scattered across directories. A path-scoped rule with a glob loads the instruction only when relevant.

Q20 - A and B

The hanging symptom points to interactive execution, so `-p` / `--print` is the direct fix. Automated downstream parsing calls for structured JSON with a schema, not regex over prose.

Q21 - B

A failing test turns refinement into an evidence-driven loop. Claude can reproduce the bug, make a focused change, and verify whether the behavior actually changed. Vague adjectives provide no objective feedback signal.

Q22 - B

The tool already succeeded, so the problem is the shape of the returned data. `PostToolUse` can normalize or annotate successful output before Claude reasons over it. A pre hook is for controlling the proposed action before execution.

Q23 - A and B

The schema currently makes genuine absence impossible, so nullable design removes pressure to fabricate `middle\_name`. Separately, schema conformance cannot prove arithmetic consistency, so invoice totals need semantic validation.

Q24 - A, B and C

Large reviews benefit from focused local passes followed by a cross-file integration pass. A fresh reviewer reduces anchoring from the generation context. Confidence can help route findings only after the threshold has been calibrated against labeled outcomes. A bigger window or same-session self-critique does not solve these failure modes.

Q25 - A and B

This workload is independent, high-volume, and non-blocking, which fits Message Batches. Batch pricing is 50% of standard API rates, processing may take up to 24 hours, and results can be out of order, so stable `custom\_id` values are essential for correlation.

SECTION 10

Exam-day quick scan

Use this as the final pass through the blueprint. If you cannot explain an item in one or two sentences, go back to that objective.

LAST-MINUTE RULE

Do not memorize the answer pattern. Memorize the condition that makes the pattern correct. "Use a hook" is not always right. It is right when the scenario needs a control at that lifecycle point.

30-objective checklist

[] 1.1 Agentic Loops	[] 1.2 Multi-Agent Orchestration
[] 1.3 Subagent Invocation and Context Passing	[] 1.4 Workflow Enforcement and Handoff
[] 1.5 Agent SDK Hooks	[] 1.6 Task Decomposition Strategies
[] 1.7 Session State and Resumption	[] 2.1 Tool Interface Design
[] 2.2 Structured Error Responses	[] 2.3 Tool Distribution and Tool Choice
[] 2.4 MCP Server Integration	[] 2.5 Built-in Tools
[] 3.1 CLAUDE.md Hierarchy, Scoping, and Modular Organization	[] 3.2 Custom Slash Commands and Skills
[] 3.3 Path-Specific Rules for Conditional Convention Loading	[] 3.4 Plan Mode vs Direct Execution
[] 3.5 Iterative Refinement Techniques	[] 3.6 CI/CD Integration
[] 4.1 System Prompts with Explicit Criteria	[] 4.2 Few-Shot Prompting
[] 4.3 Structured Output with Tool Use	[] 4.4 Validation, Retry, and Feedback Loops
[] 4.5 Batch Processing Strategies	[] 4.6 Multi-Instance and Multi-Pass Review
[] 5.1 Context Window Management	[] 5.2 Escalation and Ambiguity Resolution
[] 5.3 Error Propagation in Multi-Agent Systems	[] 5.4 Codebase Exploration and Context Degradation
[] 5.5 Human Review and Confidence Calibration	[] 5.6 Information Provenance and Multi-Source Synthesis

Five questions to ask before submitting an answer

- What exact requirement or symptom is the stem describing?
- What changed, or where is the failure actually originating?
- Does this option contain a working mechanism or only better wording?
- Does the mechanism guarantee more than the question actually asks for?
- If this is multiple response, did I select exactly the requested number?